

~~National Institute of Technology Manipur–Imphal~~
Software Engineering Life Cycle Models (L2)

Dr. Prahlada Rao B B ANRF PM Professor

31st July 2026

CSE Department

1

31-07-2026

Classical Waterfall Model

Software Life Cycle:

- Series of identifiable stages that a software product undergoes during its life time:

- Classical waterfall model divides life cycle into phases:
 - Feasibility Study,
 - Requirements Analysis And Specification, – Design,
 - Coding And Unit Testing,
 - Integration And System Testing,
 - Maintenance.

2

Classical Waterfall Model

Feasibility Study

Req. Analysis

Design

Coding

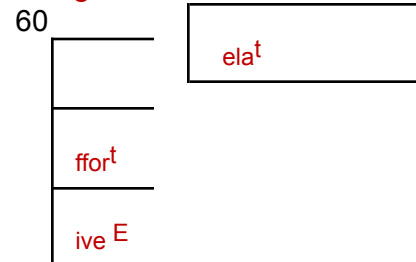
Testing

Maintenance

3

Relative Effort for Phases

- Phases between feasibility study and testing



– known as development phases. 50

- Among all life cycle phases

– Maintenance phase consumes

maximum effort.

- Among development phases,

– Testing phase consumes the maximum effort.

Req. S^p Designⁿ Codin^g Tes^t Maintnc^e

4

Classical Waterfall Model (CONT.)

- Most organizations usually define:
 - Standards on outputs (deliverables) produced at the end of every phase
 - Entry and exit criteria for every phase.
- Also prescribe specific methodologies for:
 - Specification,
 - Design,
 - Expect fresh engineers to master the organization's
 - Testing,
 - Project management, etc.

5

Classical Waterfall Model (CONT.)

- The guidelines and methodologies of an organization:
 - called the organization's Software Development Methodology.
- Software development Organizations: Software Development Methodology.

6



- **Main aim of feasibility study:** determine whether developing the product
 - Financially worthwhile
 - Technically feasible.
- **1st roughly understand what the customer wants:**
 - Different data which would be input to the system,
 - Processing needed on these data,
 - Output data to be produced by the system,
 - Various constraints on the behavior of the system. 7

Activities during Feasibility Study

- **Work out an overall understanding of the problem.**
- **Formulate different solution strategies.**
- **Examine alternate solution strategies in terms of:**
 - Resources required
 - Cost of development and
 - Development time

Activities during Feasibility Study

- Perform a cost/benefit analysis: – To determine which solution is the best.
- Determine that none of the solutions is feasible due to:
 - High cost,
 - Resource constraints,
 - Technical reasons.

9

Requirements Analysis and Specification

- Aim of this phase:
 - Understand the exact requirements of the customer,
 - Document them properly.
- Consists of two distinct activities:
 - Requirements gathering and analysis – Requirements specification.

10

Goals of Requirements Analysis

- Collect all related data from the customer:
 - Analyze collected data to Understand what the Customer Wants
 - Find out inconsistencies and incompleteness in the requirements,
 - Resolve all inconsistencies and incompleteness. • For ex. for business accounting software:

11

Requirements Gathering

- Gathering relevant data:
 - Usually collected from the end-users through interviews and discussions.
 - Interview all accountants of the organization to find out their requirements.

12

6

Requirements Analysis (CONT.)

- The data you initially collect from the users: – Would usually contain several contradictions and ambiguities:
 - Each user typically has only a partial and incomplete view of the

system.

13

Requirements Analysis (CONT.)

- Ambiguities and contradictions:
 - must be identified
 - resolved by discussions with the customers.
- Next, requirements are organized:
 - Into a software requirements specification (SRS) document.

14

7

Requirements Analysis (CONT.)

- Engineers doing requirements analysis and specification:
 - are designated as Analysts.

Design

- Design phase transforms requirements specification, into a form suitable for implementation in some programming language.
- During Design Phase, Software Architecture is derived from the SRS document.
- Two Design Approaches:
 - Traditional Approach,
 - Object Oriented Approach.

Traditional Design Approach

- Consists of two activities: – Structured Analysis
 - Structured Design

Structured Analysis Activity

- Identify all the functions to be performed. • Identify data flow among the functions.
- Decompose each function recursively into sub functions.
 - Identify data flow among subfunctions as well. • Carried out using Data Flow Diagrams (DFDs).
- After Structured Analysis, Structured Design is carried out.

Structured Design Activity

- Structured Design carried out in Two Phases: – Architectural Design (High-level Design) – Detailed Design (Low-level Design).
- High-level design:
 - Decompose the system into modules,
 - Represent invocation relationships among the modules.

- Detailed design:
 - Different modules designed in greater detail:
 - Data structures and algorithms for each module are designed.

19

Object Oriented Design

- First identify various objects (real world entities) occurring in the problem:
- Identify the relationships among the objects. For example,
- The objects in a pay-roll software may be: – Employees,
 - Managers,
 - Pay-roll register,
 - Departments, etc.

Friday

2-08-2026 Class Swapped 2

4-08-2026 Convocation

No class
20

05-08-2026

10

Object Oriented Design (CONT.)

- Object Structure
 - Further refined to obtain the detailed design.
- OOD has several advantages:
 - Lower development effort
 - Lower development time
 - Better maintainability

Implementation

- Purpose of implementation phase (Aka **Coding & Unit Testing phase**):
 - Translate software design into source code.

aka : Also Known as

Implementation- Coding, Unit Testing

- During the implementation phase:
 - Each module of the design is Coded. – Each module is Unit Tested.
 - Tested independently as stand alone unit and debugged.
 - Each module is documented.

Implementation Coding, Unit Testing

- The purpose of Unit Testing: – Test if individual modules work correctly.
- The end product of implementation phase:
 - A set of program modules that have been Tested individually.

Integration and System Testing

- Different modules are integrated in a planned manner:
 - Modules are never integrated in one shot.
 - Integration is carried out through a number of steps. • During each Integration step:
 - The partially integrated system is tested.

Integration and System Testing

System Testing

- After, all modules have been successfully integrated and tested:
 - System Testing is carried out.
- Goal of System Testing:
 - Ensure that the developed system functions according

to its requirements as specified in SRS document. 27

Maintenance

- Maintenance of any software product:
 - Requires much more effort than the effort to develop the product.
 - Development effort to Maintenance effort is typically 40:60.

28

14

Maintenance (CONT.)

- Corrective maintenance:
- Classical Waterfall Model is an idealistic one, it Assumes that no defect is
 - Correct errors which were not discovered during the product development phases.
- Perfective maintenance:
 - Improve implementation of the system
 - Enhance functionalities of the system.
- Adaptive maintenance:
 - Port Software to a new environment,
 - E.G. To a new computer or to a new operating system. introduced, and ~~Redo some of the~~ work done during that and also the

Shortcomings of Classical Waterfall Model

ever introduced during any of the lifecycle phases.

In practice:

- Defects do get introduced in almost every phase of the life cycle.
Engineers do commit errors in every phase of the life cycle.
- Reason can be due to: Oversight, Wrong Assumptions, Use of Inappropriate Technology, Communication Gap among Engineers, etc. • These Defects usually get detected much later in the life cycle:

For Example:

- A design defect might go unnoticed till coding or Testing phase. • Once a Defect is detected, Need to go back to the phase where it was

work of subsequent phases, affected by the work.

2.Iterative Waterfall Model

- Once a Defect is detected:
 - Need to go back to the phase where it was introduced Redo some of the work done, during that and all subsequent phases.
- We need feedback paths in the Classical Waterfall model.

Feasibility Study

Req. Analysis

if it is identified at end of integration and system testing

Design

Coding

Testing

Maintenance

but also to **code** and test phases. [Upto here](#)

31

Iterative Waterfall Model (CONT.)

- **Errors should be detected**

In the **same phase in which** they are **introduced**.

For example:

- **If a design problem is detected** in the **design phase itself** The **problem can be taken care of much more easily, than**

phase.

- **Reason: Rework** must be **carried out** not only to **design**,

05-08-2026

32

16

2nd ~~hr~~ Friday:
7-08-2026

Phase Containment of Errors

- The principle of **detecting errors as close to its point of introduction as possible**, is known as **phase containment of errors**.
- This is an **important Software Engineering principle**.
- **Iterative waterfall model** is by far **the most widely used model**.
- Almost **every other model is derived from the waterfall model**.

Students Left

Classical Waterfall Model (CONT.)

- Irrespective of the life cycle model followed:
 - The documents should reflect a Classical Waterfall model of development, So that The comprehension of the documents is facilitated.
- Metaphor of Mathematical Theorem Proving: – A mathematician presents a proof as a single chain of deductions, Even though the proof might have come from a convoluted set of partial attempts, blind alleys and backtracks.

Convoluted set: A set that is twisted, complicated or difficult to understand

34

17

11-08-2026

3. Prototyping Model

- Before starting actual development,
 - A working prototype of the system should first be built.
- A prototype is a toy implementation of a system with:
 - Limited functional capabilities.
 - Low reliability.
 - Inefficient performance.

Reasons for Developing A Prototype

- To Illustrate to the customer:
 - Input data formats, messages, reports, or interactive dialogs.
- Examine technical issues associated with product development: Often major design decisions depend on issues like:
 - Response time of a Hardware Controller,
 - Efficiency of a sorting algorithm, etc.
- The third reason for developing a prototype is because: It is impossible to 'get it right' the first time.
 - We must plan to throw away the first product, if We want to develop a good product.

36

18

Prototyping Model

- Start with approximate requirements. • Carry out a quick design.
- Prototype model is built using several short-cuts:
 - Short-cuts might involve using inefficient, inaccurate, – This cycle continues until the user approves the or dummy functions.
 - A function may use a table look-up rather than performing the actual computations.

37

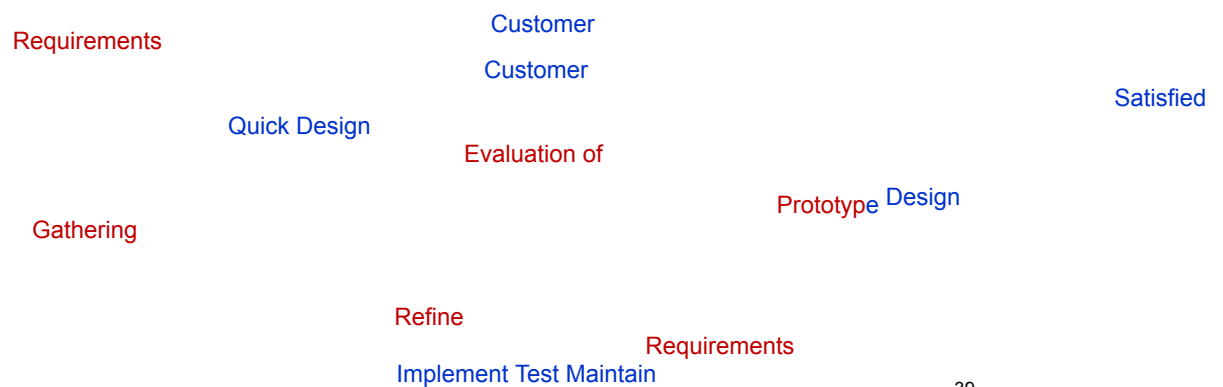
Prototyping Model (CONT.)

- The developed prototype is submitted to the customer for his evaluation:
 - Based on the user feedback, the requirements are refined.
- The actual system is developed using the classical waterfall approach.

38

19

Prototyping Model (CONT.) Build Prototype



39

Prototyping Model (CONT.)

- Requirements analysis and specification phase becomes redundant:
 - Final working prototype (with all user feedbacks incorporated) serves as an animated requirements specification.
- Design and code for prototype is usually thrown away: • However, the experience gathered from developing the prototype helps a great deal while developing the actual product.

40

20

• Many user requirements get ~~properly defined~~
Prototyping Model (CONT.)

• Even though construction of a working prototype model involves additional cost, overall development cost might be lower for:

- Systems with unclear user requirements,
- The system is broken down into several modules which
- Systems with unresolved technical issues. • ~~First develop the core modules~~ of the system.

and technical issues get resolved:

- These would have appeared later as change requests and resulted in incurring massive redesign costs.

- By adding new functionalities in successive versions. ~~Aka—Also Known as~~

4. Evolutionary Model

- Evolutionary model (Also Known as successive versions or incremental model):

can be incrementally implemented and delivered.

- The initial product skeleton is refined into increasing levels of capability:

42

21

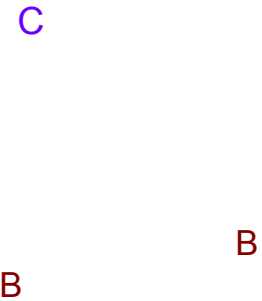
Evolutionary Model (CONT.)

- Successive version of the product: – Functioning systems capable of performing some useful work.
- A new release may include new functionality:
 - Also existing functionality in the current release might have been enhanced.

43

Evolutionary Model (CONT.)

Fig 2.6 Evolutionary Development of Software Product



A, B, C are modules of Software Product that are Incrementally Developed and delivered ⁴⁴

22

Advantages of Evolutionary Model

- Users get a chance to experiment with a partially developed system:
 - Much before the full working version is released, • Helps finding exact user requirements: – Much before fully working system is developed. • Core modules get tested thoroughly: – Reduces chances of errors in final product.

45

Disadvantages of Evolutionary Model

- Often, difficult to subdivide problems into functional units:
 - That can be incrementally implemented and delivered.

- Evolutionary model is useful for very large problems,
 - Where it is easier to find modules for incremental implementation.

46

23

5. Evolutionary Model with Iteration

- Many organizations use a combination of Iterative And Incremental development:
 - A new release may include new functionality
 - Existing functionality from the current release may also have been modified.

47

Evolutionary Model with iteration

Several advantages:

- Training can start on an earlier release
 - Customer feedback taken into account
- Markets can be created:

- For functionality that has never been offered.
- Frequent releases allow developers to fix unanticipated problems quickly.

48

24

6. Spiral Model

- Proposed by Boehm in 1988.
- Each loop of the spiral represents a phase of the software process:
 - The innermost loop might be concerned with system feasibility,
 - The next loop with system requirements definition, – The next one with system design, and so on.
- There are no fixed phases in this model, the phases shown in figure are just examples.

49

Spiral Model (CONT.)

- The team must decide:
 - How to structure the project into phases.
- Start work using some generic model: add extra phases
 - For specific projects or when problems are identified during a project.
- Each loop in the spiral is split into four sectors (quadrants).

Spiral Model



Fig. 2.2: Spiral Model

51

Spiral Model (CONT.)

Resolve Risks

Determine
Objectives

Develop Next
Level of Product 52

Customer
Evaluation of Prototype
Identify &

Objective Setting (First Quadrant)

- Identify objectives of the phase,
- Examine the risks associated with these objectives.
 - Risk: Any adverse circumstance that might hamper successful completion of a software project.
- Find alternate solutions possible.

53

Risk Assessment and Reduction (2nd Quadrant)

- For each identified project risk,
 - A detailed analysis is carried out.
- Steps are taken to reduce the risk. For example:
- If there is a risk that the requirements are inappropriate:
 - A prototype system may be developed.

54

- **Development and Validation (Third Quadrant):** – Develop and validate the next level of the product.
- **Review and Planning (Fourth Quadrant):**
 - Review the results achieved so far with the customer and plan the next iteration around the spiral.
- **With each iteration around the spiral:** – Progressively more complete version of the

11-08-2026

55

Wed
12-08-2026

Spiral Model as a Meta Model

- **Subsumes all discussed Models:** – A single loop spiral represents waterfall model. Uses an evolutionary approach
 - Iterations through the spiral are evolutionary levels.
 - Enables understanding and reacting to risks during each iteration along the spiral.
- **Uses:**
 - Prototyping as a risk reduction mechanism
 - Retains the step-wise approach of the waterfall model.

56

2. Iterative waterfall model

- Most widely used model.

– But, suitable only for well-understood problems.

3. Prototype model is suitable for projects not well understood:

- User requirements
- Technical aspects

57

Comparison of Different Life Cycle Models (CONT.)

4. Evolutionary Model is suitable for large problems:– Can be decomposed into a set of modules that can be incrementally implemented,

- Incremental delivery of the system is acceptable to the customer.

5. The Spiral Model:

- Suitable for development of technically challenging software products that are subject to several kinds of risks.

58